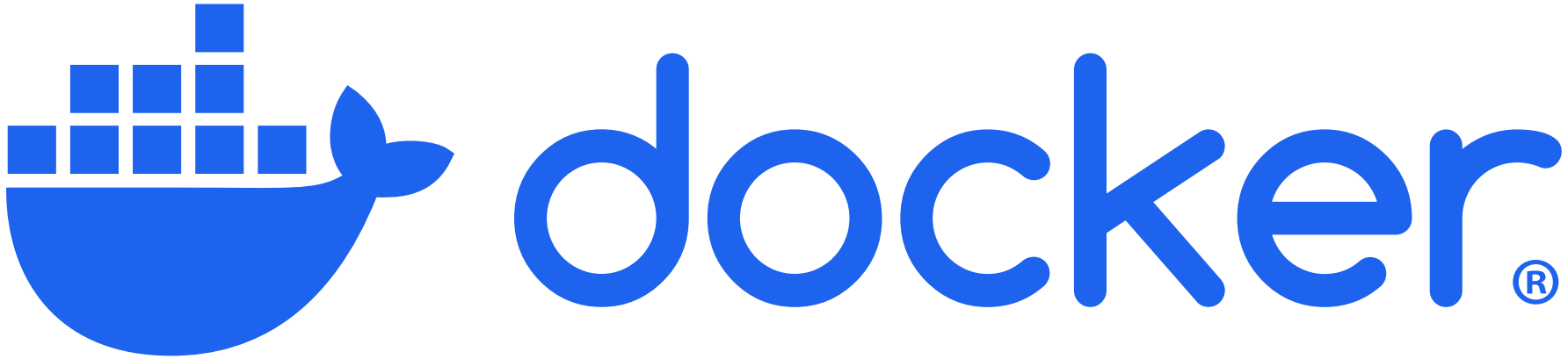
# Formação Completa

# AWS Developer Associate

# Criação e Manutenção de APIs - Parte 2






# O que é Docker?

### Plataforma de Containerização

O Docker empacota aplicações e dependências em contêineres isolados e portáteis.

### Execução Consistente

Garante que aplicações rodem da mesma forma em diferentes ambientes.

### Essencial para AWS

Base para serviços como ECS e EKS, facilitando implantações em nuvem.

### Leve e Eficiente

Consome menos recursos que máquinas virtuais tradicionais.

# Vantagens do Docker

1. Previsibilidade: Elimina surpresas durante o deployment e problemas de compatibilidade entre ambientes.
2. Portabilidade Universal: Funciona de forma idêntica em qualquer máquina, do desenvolvimento à produção.
3. Eficiência Operacional: Reduz significativamente o trabalho necessário para gerenciar e implantar aplicações.
4. Flexibilidade Tecnológica: Suporta qualquer linguagem, sistema operacional e tecnologia de sua escolha.
5. Ideal para Microsserviços: Facilita a migração de aplicações locais para a nuvem AWS.

# Onde as imagens do Docker são armazenadas?

## Docker Hub

Repositório público oficial do Docker.

- Acesso gratuito a milhares de imagens
- Contém imagens base (Ubuntu, MySQL, Node.js)
- Integração nativa com CLI do Docker

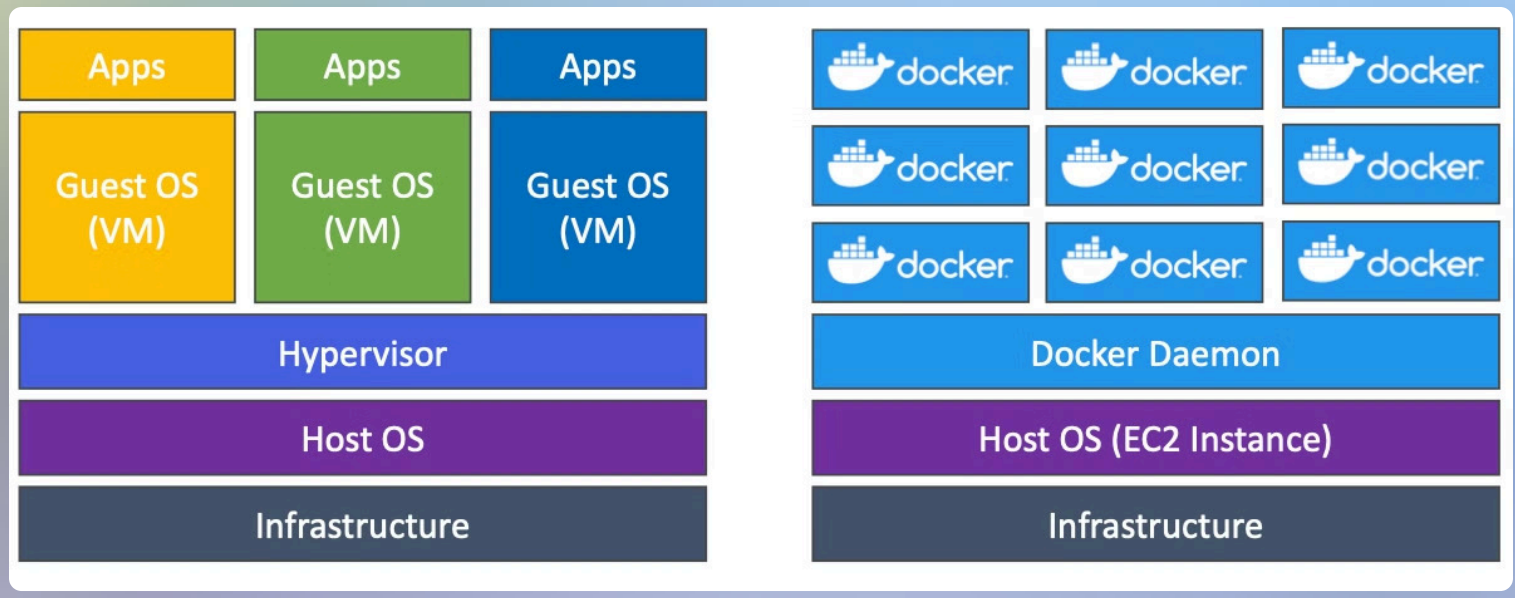
Acesse em: **hub.docker.com**

## Amazon ECR

Repositório gerenciado pela AWS para armazenar imagens privadas.

- Segurança integrada com IAM
- Replicação entre regiões
- Escaneamento automático de vulnerabilidades

Também oferece galeria pública: **gallery.ecr.aws**




# Docker vs. Máquinas Virtuais

## Docker

Não é uma virtualização completa, mas sim uma containerização.

- Compartilha o kernel do sistema host
- Inicia em segundos
- Tamanho reduzido (MBs)
- Baixo overhead de recursos
- Isolamento a nível de processo

## Máquinas Virtuais

Virtualização completa com sistemas operacionais isolados.

- Sistema operacional completo por VM
- Inicia em minutos
- Tamanho maior (GBs)
- Maior consumo de recursos
- Isolamento total de hardware

## Casos de Uso

Docker: Ideal para microsserviços e aplicações modernas.

VMs: Preferível para isolamento completo e cargas de trabalho legadas.

Na AWS: Docker é recomendado para serviços como ECS e EKS.



# Amazon ECS - Elastic Container Service

## O que é

Serviço gerenciado que simplifica o deploy e a escalabilidade de aplicações containerizadas.

Elimina a necessidade de gerenciar a infraestrutura subjacente.

## Vantagens

- Fácil integração com outros serviços AWS
- Escalabilidade automática
- Gerenciamento simplificado

# Amazon EKS - Elastic Kubernetes Service

## O que é

Serviço gerenciado AWS para implementação e orquestração de clusters Kubernetes sem gerenciar os control planes.

## Arquitetura

Gerencia automaticamente disponibilidade e escalabilidade dos nós Kubernetes em múltiplas zonas de disponibilidade.

## Segurança

Integração com serviços AWS IAM para autenticação, além de criptografia e isolamento de rede.

## Certificado

Certificado pela Cloud Native Computing Foundation, garantindo compatibilidade com ecossistema Kubernetes padrão.

Ideal para organizações que precisam de Kubernetes gerenciado com flexibilidade para personalização avançada e aplicações distribuídas geograficamente.

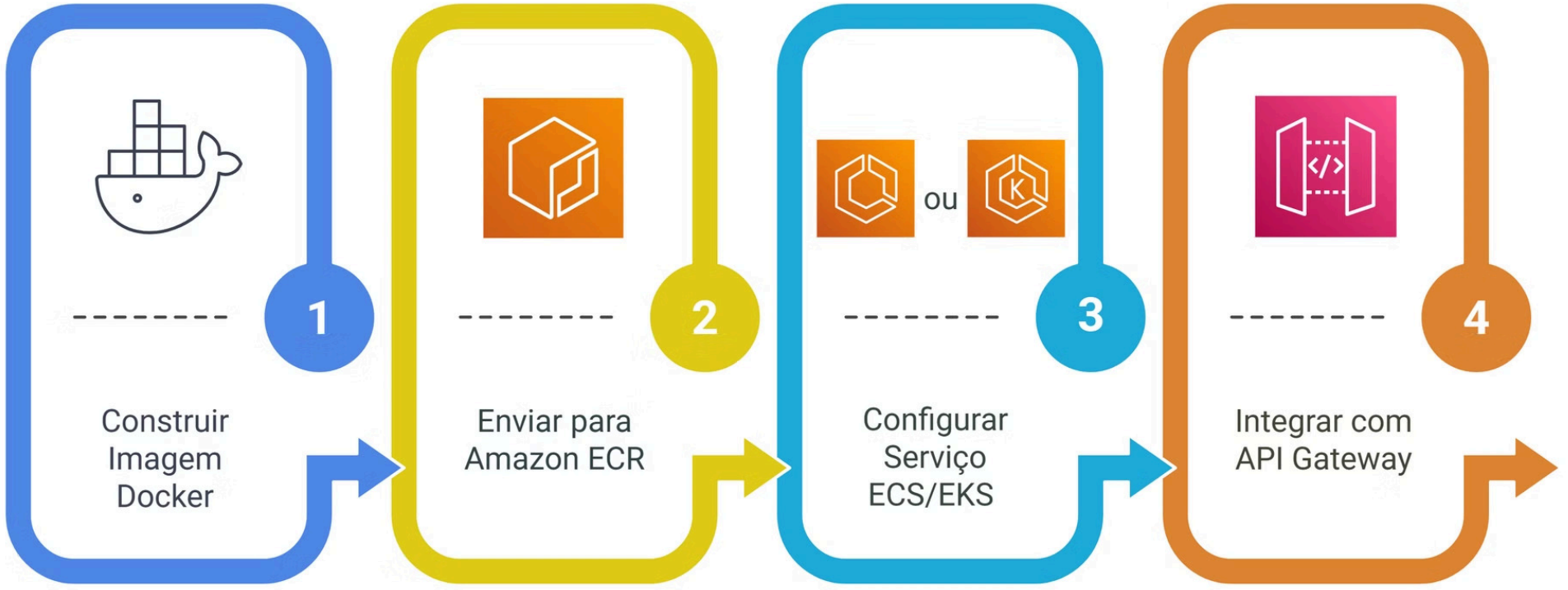





# Comparação: ECS vs EKS

| Característica | Amazon ECS | Amazon EKS |
|---|---|---|
| Complexidade | Menor | Maior |
| Flexibilidade | Boa | Excelente |
| Curva de aprendizado | Suave | Íngreme |
| Integração AWS | Nativa | Boa |

# Implantação de Contêineres



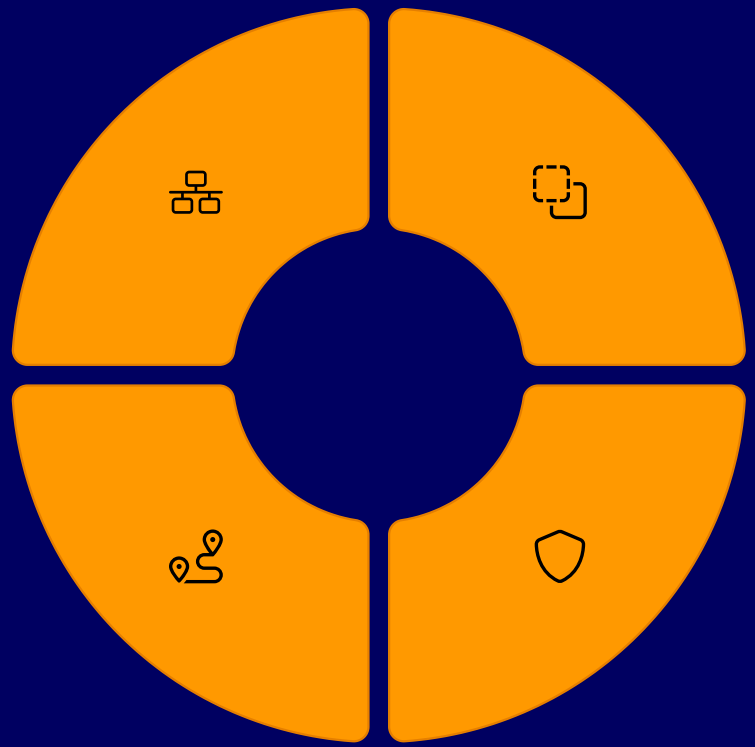
# Considerações de Networking

## VPC

Configure sua Virtual Private Cloud para isolamento de rede.

## Sub-redes

Distribua recursos em sub-redes públicas e privadas.



## Tabelas de Rotas

Configure o roteamento de tráfego entre sub-redes.

## Grupos de Segurança

Defina regras de firewall para controlar o tráfego.



# Gestão de Custos

**24/7**

### Monitoramento

Acompanhe o uso de recursos continuamente.

**30%**

### Economia Potencial

Redução de custos com dimensionamento adequado.

**$$$**

### Complexidade

Custos podem aumentar com a complexidade da infraestrutura.

# Exemplo de Integração ECS com API Gateway

```yaml
Resources:
  MyContainerApi:
    Type: AWS::Serverless::Api
    Properties:
      StageName: Prod
      DefinitionBody:
        openapi: "3.0.1"
        info:
          title: ContainerAPI
          version: "1.0"
        paths:
          /service:
            get:
              x-amazon-apigateway-integration:
                type: http_proxy  # Para integrar o API Gateway a um serviço containerizado no ECS
                uri: "http://<seu-load-balancer-dns>/service"  # Endpoint HTTP do seu serviço
                httpMethod: GET  # Ou POST, dependendo da sua API
```

**Escolha um tipo de integração de API do API Gateway**: https://docs.aws.amazon.com/pt_br/apigateway/latest/developerguide/api-gateway-api-integration-types.html




# Benefícios dos Contêineres

### Escalabilidade

Aumente ou diminua recursos conforme a demanda.

### Isolamento

Cada aplicação opera em seu próprio ambiente.

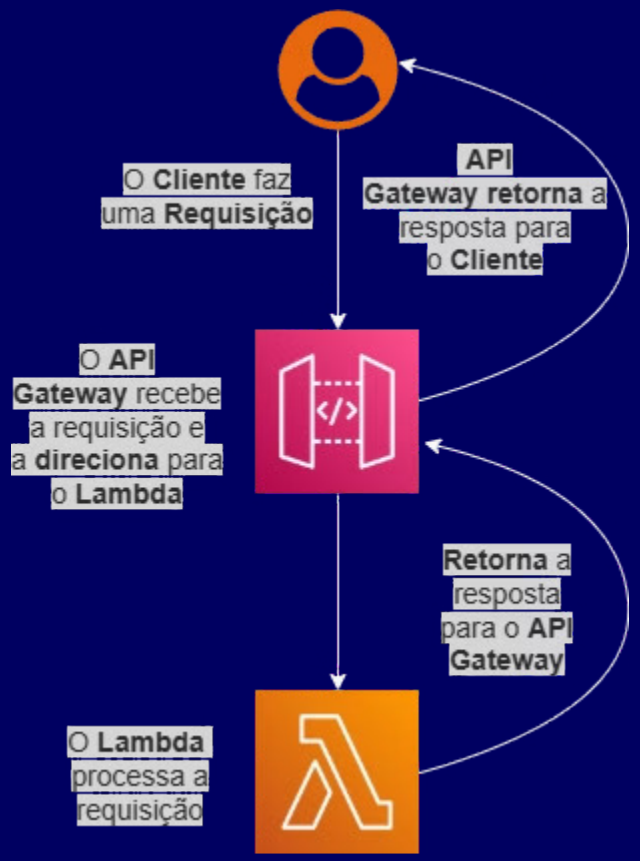
### Consistência

Mesmo comportamento em desenvolvimento e produção.

### Eficiência

Inicialização rápida e baixo overhead.

# Integração com Lambda e API Gateway




### Cliente

Faz requisição HTTP para um endpoint.

### API Gateway

Recebe e direciona a requisição.

### Lambda/ECS/EKS

Processa a requisição e retorna resposta.

### Resposta

Retorna ao cliente via API Gateway.

# Versionamento no AWS Lambda

**Desenvolvimento**

Criação ou atualização da função Lambda.

**Publicação de Versão**

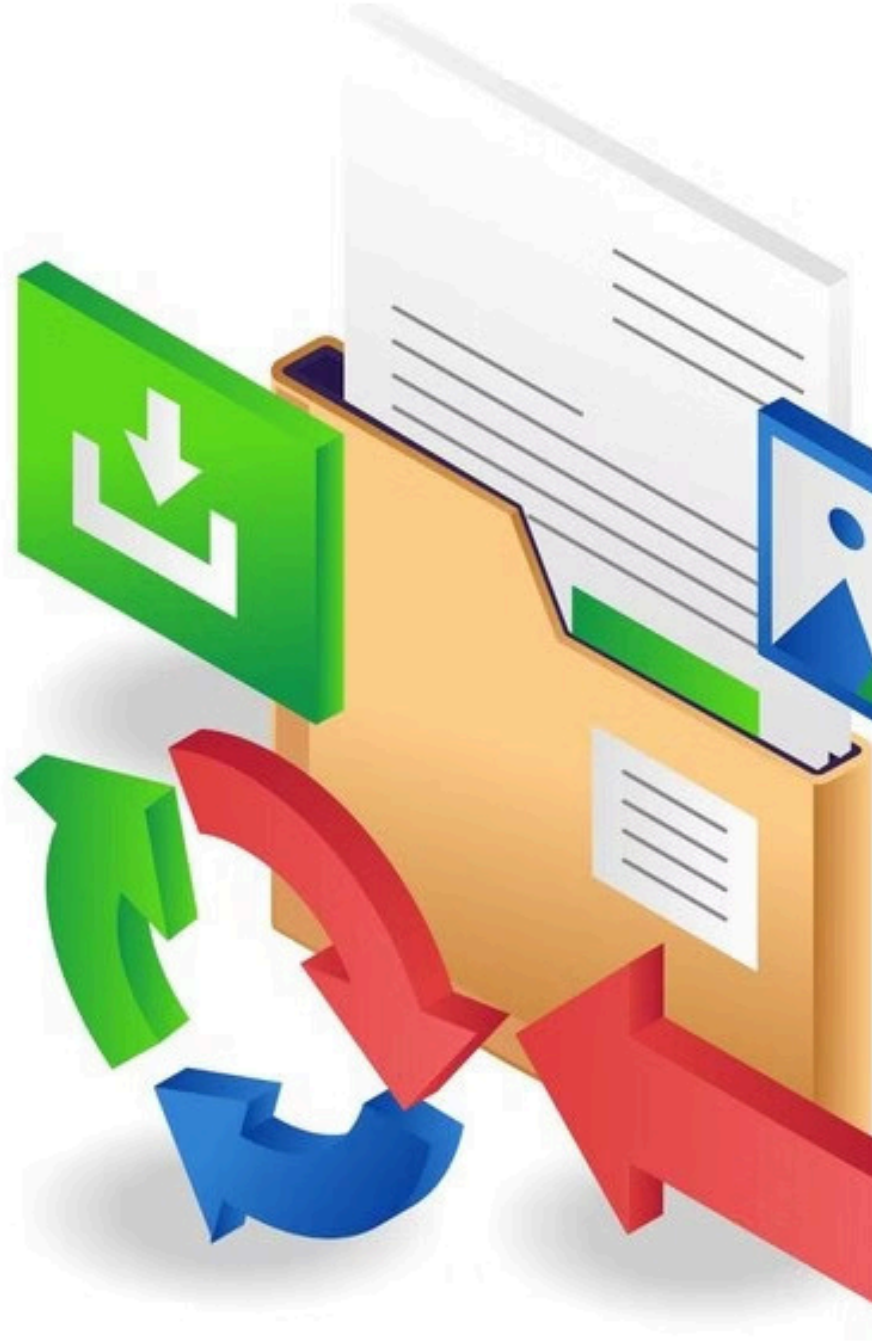
Snapshot imutável do código e configuração.

**Criação de Alias**

Referência para versão específica.

**Atualização**

Modificação do alias para apontar para nova versão.



# Benefícios do Versionamento



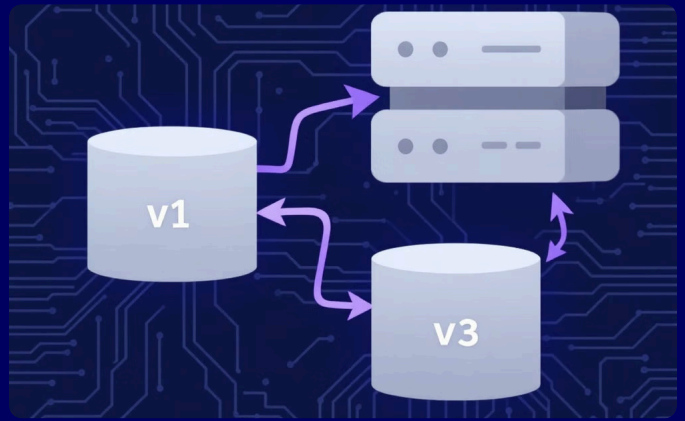

## Imutabilidade

Versões publicadas não podem ser alteradas, garantindo consistência.




## Rollback Fácil

Retorne rapidamente a versões anteriores em caso de problemas.




## Implantação Gradual

Direcione porcentagens do tráfego para diferentes versões.

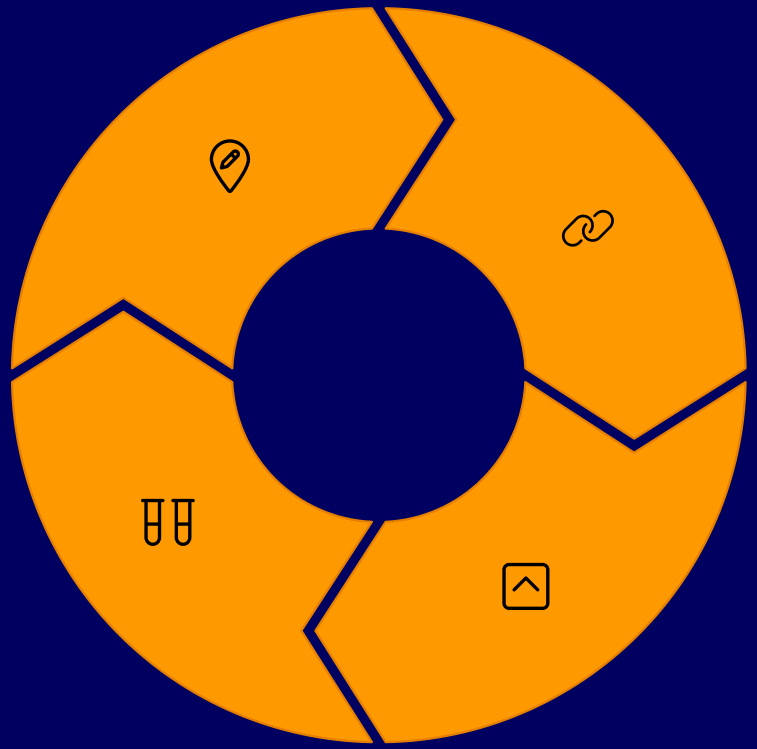
# Aliases no AWS Lambda

**Criação**

Defina um alias apontando para uma versão específica.

**Validação**

Verifique o funcionamento da nova versão.



**Integração**

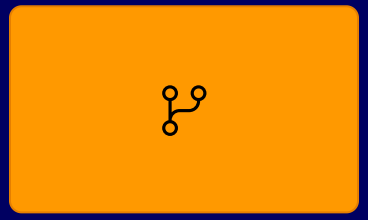
Use o alias em vez da versão nas integrações.

**Atualização**

Modifique o alias para apontar para nova versão.

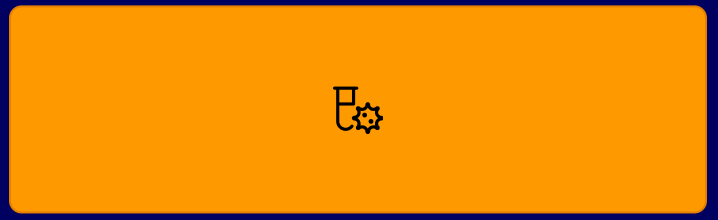
# Estágios (Stages) no Amazon API Gateway

No **API Gateway**, um **Stage** (estágio) representa uma **versão implantada** da API. Ele permite gerenciar diferentes ambientes, como **desenvolvimento (dev), teste (staging) e produção (prod)**, sem precisar recriar toda a API.



### Desenvolvimento

Ambiente para testes durante o desenvolvimento ativo.



### Teste

Ambiente para validação formal antes da liberação.



### Produção

Ambiente final acessado pelos usuários reais.

# Exemplo de Configuração de Stage

## Exemplo de Stages no API Gateway (YAML - AWS SAM)

```yaml
Resources:
  MyApi:
    Type: AWS::Serverless::Api
    Properties:
      StageName: Prod  # Define o estágio como "Prod"
```

## Se precisar criar vários stages manualmente

```yaml
Resources:
  MyApi:
    Type: AWS::ApiGateway::Stage
    Properties:
      StageName: Dev
      RestApiId: !Ref MyApi
      DeploymentId: !Ref MyApiDeployment
```

## Dessa forma, você pode acessar a API em URLs diferentes, como:

**Dev:** https://abc123.execute-api.us-east-1.amazonaws.com/Dev/

**Prod:** https://abc123.execute-api.us-east-1.amazonaws.com/Prod/

# Benefícios da Abordagem de Stages e Aliases



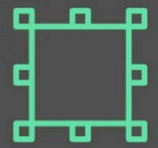
## Isolamento de Ambientes

Segregação clara entre desenvolvimento, teste e produção.



## Facilidade de Rollback

Reversão simples para versões anteriores em caso de problemas.



## Flexibilidade na Integração

Atualizações dinâmicas sem reconfiguração manual dos endpoints.

# Monitoramento e Observabilidade

## Para recordarn💭

### CloudWatch Logs

Registros detalhados de execução das funções Lambda e requisições ao API Gateway.

- Análise de erros
- Depuração de problemas
- Auditoria de acessos

### CloudWatch Metrics

Métricas de desempenho e utilização.

- Latência de requisições
- Taxa de erros
- Número de invocações
- Consumo de memória

### X-Ray

Rastreamento distribuído para análise de performance.

- Visualização de chamadas
- Identificação de gargalos
- Mapa de serviços

# Melhores Práticas

### Segurança

- Use IAM roles com privilégios mínimos
- Implemente autenticação e autorização
- Criptografe dados sensíveis

### Performance

- Otimize o código das funções Lambda
- Configure memória adequadamente
- Use caching quando possível

### Operação

- Implemente CI/CD automatizado
- Configure alertas para anomalias
- Documente APIs com OpenAPI



# Por que o que vimos até aqui sobre Criação e Manutenção de APIs é importante?

# (Visão Prática e da Certificação)

# APIs na AWS: Pontos Cruciais para Certificação

## Validação de Payload

Filtra dados maliciosos e garante requisições válidas. Fundamental para segurança e integridade dos dados.

## Transformação de Dados

Adapta formatos entre sistemas diferentes. Essencial para integração eficiente entre aplicações diversas.

## Códigos de Status

Comunicam claramente resultados das operações. Facilitam depuração e melhoram experiência do usuário.

## Manutenção Simplificada

Boas práticas reduzem complexidade e custos. Permitem evolução contínua sem interrupções de serviço.

# Testes Automatizados



## Verificação Automática

Ferramentas como Postman, cURL e SAM local automatizam testes de funcionalidade e desempenho das APIs.



## Prevenção de Regressões

Testes consistentes reduzem o risco de falhas em produção após alterações no código.



## Garantia de Qualidade

A validação sistemática aumenta a confiabilidade e robustez das interfaces.



## Integração Contínua

Testes automatizados são essenciais para pipelines CI/CD eficientes na AWS.

# Integração com Serviços AWS



## Integração Completa

API Gateway trabalha perfeitamente com Lambda, ECS/EKS e ferramentas de CI/CD como CodePipeline.



## Automação de Processos

Deploy e manutenção automatizados garantem consistência entre ambientes de desenvolvimento, teste e produção.



## Segurança Integrada

Integrações automatizadas asseguram proteção e escalabilidade em todos os níveis da arquitetura.



## Adaptabilidade

Sistemas robustos que respondem rapidamente a mudanças sem comprometer desempenho.

# Qual a importância para o mercado?

### Inovação Acelerada

APIs e contêineres reduzem o tempo de lançamento de novas soluções.

### Integração de Ecossistemas

Facilita parcerias estratégicas e amplia ofertas de serviços.

### Conformidade e Segurança

Atende requisitos regulatórios com controles padronizados.

### Redução de Custos Operacionais

Implementações eficientes geram economia significativa de recursos.

O domínio dessas tecnologias na AWS não apenas atende demandas técnicas, mas também posiciona profissionais estrategicamente no mercado atual.

Empresas buscam constantemente otimizar arquiteturas de microserviços para atender objetivos de negócio com máxima eficiência.